17 DE DICIEMBRE DE 2025

MEMORIA DE PROGRAMACIÓN

INTERFAZ GRÁFICA DE ANÁLISIS Y PROCESADO DE SEÑALES

VÍCTOR MERINO DE LA CALLE Y ANA SANTIAGO PRIETO

Contenido

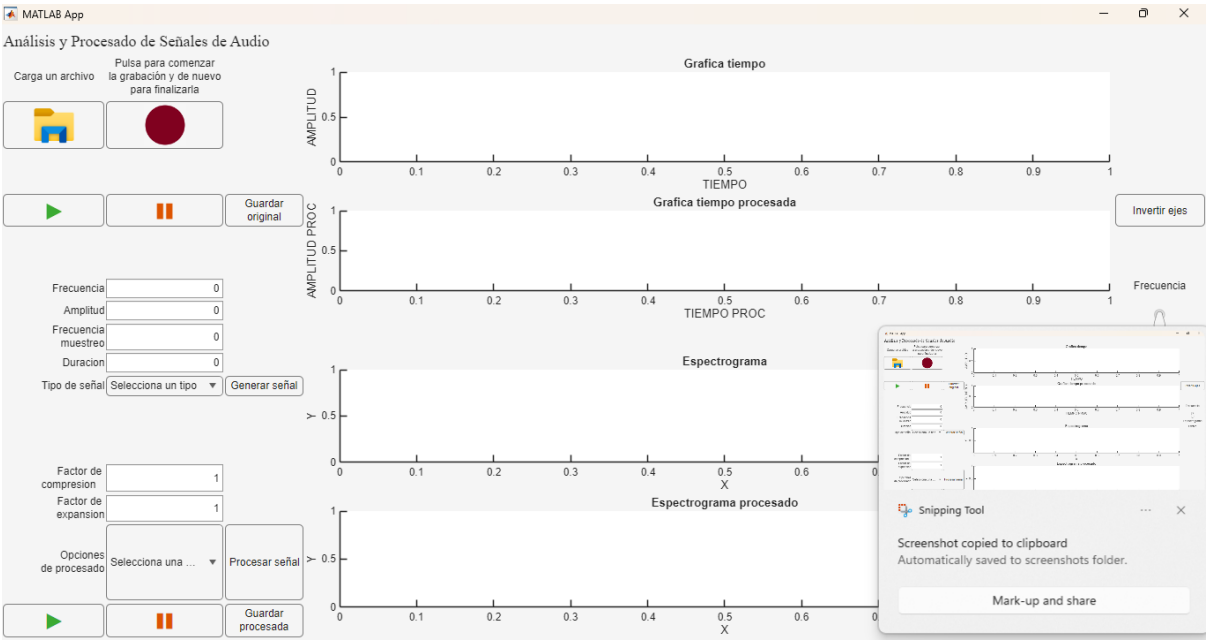
1. Introducción 2
2. Conceptos básicos 2
   a. Propiedades 2
   b. Propiedades provenientes de la aplicación 3
3. Funciones 4
   a. Carga de señales 4
   b. Generación de tonos 6
   c. Grabación mediante micrófono 8
   d. Reproducción de señales originales 9
   e. Representación de señales originales 10
      i. Tiempo 11
      ii. Espectrograma 11
4. Procesado de señales 12
   a. Compresión 13
   b. Expansión 13
   c. Compresión y expansión 13
   d. Reproducción de señales procesadas 14
   e. Representación de señales procesadas 15
      i. Tiempo 16
      ii. Espectrograma 16
5. Exportación de señales 17
   a. Exportación de señales originales 17
   b. Exportación de señales procesadas 18
6. Inversión de ejes 18
7. Cambio de gráfica (Espectrograma – Frecuencia) 19

1. Introducción

Esta memoria está destinada a explicar el diseño en lo que se refiere a la programación de nuestra interfaz que permite el análisis y el procesado de señales.

En la propia Memoria de Usuario, se hablará de cómo utilizar la interfaz. Es una guía acompañada de ejemplos explicativos para poder acceder de manera sencilla a todas las funcionalidades que esta posee.

Este documento, sin embargo, se centra en los aspectos más específicos acerca de la programación a través de la cual se aplican dichas funcionalidades.



2. Conceptos básicos

a. Propiedades

Empezaremos explicando las propiedades que manipulamos en las distintas funciones del programa.

Cada señal a manipular tiene una función (la propia señal), se muestrea a una cierta frecuencia, y debe permitir la posibilidad de ser procesada.

Partimos de una señal original (***y_original***), que puede ser una señal grabada mediante micrófono, cargada desde archivos, generada con unas características a elegir, y finalmente, representada y reproducida.

Para poder tanto representarla como reproducirla en MATLAB, debemos poder digitalizarla. Esto se lleva a cabo mediante el proceso de muestreo, que a su vez requiere una frecuencia de muestreo (***Fs***) para especificar el número de muestras y el periodo con el que se toman.

Creamos una variable llamada ***objgr*** que utilizaremos en la función que permite a la interfaz implementar la grabación de audios. En ella almacenaremos el objeto grabador (***audiorecorder***). Para saber si estamos grabando en un instante, cremoas ***IsRecording***, que se inicializa en false.

Dado que la interfaz permite implementar procesado de señales, debemos tener una propiedad donde podamos guardar la señal procesada (***y_procesada***), que también deberemos digitalizar para poder representarla (la muestrearemos con ***Fs_procesada***).

Para la reproducción de señales, cremos ***player_original*** y ***player_procesada*** que son variables donde almacenaremos objetos de reproducción. También creamos ***IsAudioReversed*** que se inicializa en falso y servirá para poder reproducir la señal al revés cuando invertimos los ejes.

Código:

```
    properties (Access = private)

        %Variables para guardar información sobre la señal original
        y_original;         %vector de la señal
        Fs;                 %frecuencia a la que muestreamos

        %Variables para la grabación
        IsRecording (1, 1) logical = false; %variable para saber si grabamos o no
        objgr;              %objeto donde guardamos la señal grabada

        %Variables para guardar información sobre la señal procesada
        y_procesada;        %vector de la señal
        Fs_procesada;       %frecuencia a la que muestreamos

        %Variables para la reproducción de señales
        IsAudioReversed (1, 1) logical = false; %Reproducir al revés
        player_original;    %reproducción de la señal original
        player_procesada;   %reproducción de la señal procesada

    end
```

b. Propiedades provenientes de la aplicación

Estas propiedades las crea MATLAB a partir de los componentes que utilizamos en la interfaz. Así, cuando el usuario utiliza los elementos, el programa responde siguiendo el código donde están implementados.

Tenemos siete botones. Cuatro de ellos se utilizan para la reproducción de los audios (***PlayButton***, ***PauseButton***, ***PlayButtonProcesada***, ***PauseButtonProcesada***). De los otros tres, que se utilizan para interactuar con el explorador de archivos del ordenador, uno es para cargar una señal que tengamos (***CargarButton***), y los otros dos para guardar las señales que hemos generado (***GuardarButton, GuardarProcesadaButton***).

La interfaz cuenta con dos menús desplegables para seleccionar tipo de señal a generar y tipo de procesado que se le quiere aplicar a una señal. Son ***tipoDropDown*** y ***opcionesdeprocesado***, respectivamente.

Por último, tenemos los campos numéricos. Para la generación de señales hay cuatro: ***frecuenciaField***, ***amplitudField***, ***frecmuestrField*** y ***duracionField***. Para el procesado hay dos: ***factordeexpansion*** y ***factordecompresion***.

El resto de las variables son “Labels”, es decir, etiquetas de campos que rellenar o seleccionar, y “Axes”, para la representación de las gráficas.

Código:

```
    properties (Access = public)
        UIFigure                    matlab.ui.Figure
        GridLayout                  matlab.ui.container.GridLayout
        Switch                      matlab.ui.control.ToggleSwitch
        SwitchLabel                 matlab.ui.control.Label
        opcionesdeprocesado         matlab.ui.control.DropDown
        OpcionesdeprocesadoLabel    matlab.ui.control.Label
        duracionField               matlab.ui.control.NumericEditField
        DuracinLabel                matlab.ui.control.Label
        factordeexpansion           matlab.ui.control.NumericEditField
        FactordeexpansionLabel      matlab.ui.control.Label
        factordecompresion          matlab.ui.control.NumericEditField
        factordeLabel               matlab.ui.control.Label
        frecmuestrField             matlab.ui.control.NumericEditField
        FrecuenciamuestreoLabel     matlab.ui.control.Label
        amplitudField               matlab.ui.control.NumericEditField
        AmplitudLabel               matlab.ui.control.Label
        frecuenciaField             matlab.ui.control.NumericEditField
        FrecuenciaLabel             matlab.ui.control.Label
        TipoDropdown                matlab.ui.control.DropDown
        TipoLabel                   matlab.ui.control.Label
        GenerarsealButton           matlab.ui.control.Button
        ProcesarsealButton          matlab.ui.control.Button
        InvertirejesButton          matlab.ui.control.Button
        Pulsaparacomenzarlagrabacinydenuevoparafinalizarla Label
matlab.ui.control.Label
        CargaunarchivoLabel         matlab.ui.control.Label
        GrabarButton                matlab.ui.control.Button
        PauseButtonProcesada        matlab.ui.control.Button
        PlayButtonProcesada         matlab.ui.control.Button
        PauseButton                 matlab.ui.control.Button
        PlayButton                  matlab.ui.control.Button
        GuardarProcesadaButton      matlab.ui.control.Button
        GuardaroriginalButton       matlab.ui.control.Button
        Button                      matlab.ui.control.Button
        AnlisisyProcesadodeSealesdeAudio  matlab.ui.control.Label
        TiempoAxes                  matlab.ui.control.UIAxes
        GraficaTiempoProcesada      matlab.ui.control.UIAxes
        Espectrograma               matlab.ui.control.UIAxes
        EspectrogramaProcesado      matlab.ui.control.UIAxes
    end
```

3. Funciones

a. Carga de señales

Este apartado muestra cómo se realiza la carga de señales desde los archivos del ordenador. La función comienza cuando se pulsa el icono del explorador de archivos en la interfaz (encima de él hay una etiqueta donde pone “Carga un archivo”).

Definimos unas variables ***filename*** y ***pathname*** que almacenan el nombre y la ruta, respectivamente, del archivo que seleccionamos al desplegar la ventana del explorador. Con la condición de que alguna de las dos variables sea 0, comprobamos si el usuario pulsó la opción de cancelar.

Creamos otra variable (***fullFilePath***) en la que recogemos el archivo entero mediante la función de MATLAB ***fullfile*** a la que le damos como atributos ***pathname*** y ***filename***.

Una vez tenemos el audio, lo leemos dentro de un try. Si no se genera ningún error, se almacena en ***y_original*** y en ***Fs*** la pista de audio y su frecuencia de muestreo, respectivamente. Además, vaciamos la variable que contiene el objeto reproductor (***player_original***).

Si se genera un error, sale por pantalla una ventana en la que se especifica cuál ha sido el error generado.

Por último, si la señal recogida es estéreo (tiene más de una dimensión), la convertimos en mono mediante la función ***mean***, actualizamos ***y_original*** y llamamos a la función ***actualizarGraficasOriginales()***. Si no es estéreo (directamente es mono), llamamos a ***actualizarGraficasOriginales()***.

```
        function ButtonPushed(app, event)
        %La función empieza cuando se pulsa "Cargar"

            %Ventana de selección de fichero
            [filename, pathname] = uigetfile({'*.wav';'*.mp3';'*.wma'},
'Seleccionar archivo de audio');

            %Comprobación de "Cancelar"
            if isequal(filename, 0) || isequal(pathname, 0)
                return;
            end

            %Creo una variable para el audio (ruta y nombre)
            fullFilePath = fullfile(pathname, filename);

            try
                %Recojo el audio (señal y frecuencia de muestreo)
                [app.y_original, app.Fs] = audioread(fullFilePath);

                %Vacío la variable player_original para poder reproducir esta
                app.player_original = [];
            catch ME
                %Si el archivo no se puede leer, lo indico
                uialert(app.UIFigure, ['Error al leer el archivo: ' ME.message],
'Error de Archivo');
                return;
            end

            if size(app.y_original, 2) > 1
                %Convierto de estéreo a mono si es el caso
                app.y_original = mean(app.y_original, 2);

                %Grafico la señal
                app.actualizarGraficasOriginales();
            else
                %Grafico la señal
                app.actualizarGraficasOriginales();
            end

        end
```

b. Generación de tonos

La función comienza cuando se pulsa “Generar señal”.

Definimos ***Fs*** como la frecuencia de muestreo que se haya insertado en el campo destinado a ello de la interfaz.

Después, obligamos a que los valores que se hayan elegido sean mayores que 0 (amplitud, frecuencia y frecuencia de muestreo). Además, para no cometer aliasing, ***Fs*** debe ser como mínimo el doble de la frecuencia de la señal, por lo que, si eso no se cumple, salta una alerta que indica que con esa frecuencia se produce este fenómeno.

Si todas las comprobaciones anteriores son correctas, se define un vector de tiempo (***tie***) que va de 0 hasta la duración total que se haya elegido (el usuario la indicará en el campo llamado ***Duración*** en la interfaz) en intervalos del periodo de muestreo (1/***Fs***). También se define una variable ***w***=2π***frecuenciaField.Value*** (***frecuenciaField.Value*** es la frecuencia de la señal generada) que son los rad/s y se utilizará en cada función.

Hacemos un switch para elegir la función que vamos a generar, que dependerá del valor que tenga la caja de selección de función. Podrá ser seno, coseno, cuadrada, triangular o sierra. Para seno y coseno, decimos que ***y_original*** es igual al parámetro de la amplitud que recibimos de la interfaz multiplicado por las funciones de Matlab ***sin*** y ***cos*** (respectivamente), que llevan como argumento ***w*tie***. Para triangular, multiplicamos la amplitud por ***sawtooth***, que tiene como argumentos ***w*tie*** y 0.5 (respectivamente). Este último parámetro es la duración de la subida de la señal, de forma que la triangular utiliza la mitad de tiempo para subir y la otra mitad para bajar, y la sierra solo sube durante todo el tiempo y baja de golpe.

Para la señal cuadrada,

Para la señal sierra,

Por último, una vez ejecutado el caso especifico que se haya seleccionado, se vacía el objeto reproductor (***player_original***) y se grafica la señal resultante generada (***y_original***) llamando a ***actualizarGraficasOriginales()***.

Código:

```
        function GenerarsealButtonPushed(app, event)
        %Comienza cuando se pulsa "Generar señal"

            %Carga la frecuencia de muestreo que el usuario introduce
               app.Fs = app.frecmuestrField.Value;

            %Si no se rellena algún parámetro o no es válido, se indica
               if (app.frecuenciaField.Value <= 0 || app.amplitudField.Value <= 0
|| app.Fs <=0 || app.duracionField <= 0)
                     uialert(app.UIFigure, 'define parametros validos (mayores
que 0)');
                     return;
               end

            %Si no se cumple la condición de Nyquist, se indica
               if (app.frecuenciaField.Value > app.Fs/2)
                     uialert(app.UIFigure, 'la frecuencia seleccionada produce
aliasing');
                     return;
```

```
            end

        %Se define un vector de tiempo
            tie = (0:1/app.Fs:app.duracionField.Value - 1/app.Fs);

        %Se define la frecuencia en radianes
        w = 2*pi*app.frecuenciaField.Value;

        %Switch de selección de función
        switch app.TipoDropdown.Value

                 case 'Seno'
                        app.y_original = app.amplitudField.Value*sin(w*tie);

                 case 'Coseno'
                        app.y_original = app.amplitudField.Value*cos(w*tie);

                 case 'Cuadrada'
                        app.y_original = zeros(size(tie));

                % Sumamos solo los primeros 20 armónicos impares (1, 3, 5...)
                % Cuantos más sumes, más cuadrada será, pero si te pasas de
Fs/2 aparecerá aliasing.
                num_armonicos = 20;

                for k = 1:2:(num_armonicos*2)
                    % Formula de Fourier: Amplitud * sin(k*w*t) / k
                    app.y_original = app.y_original + (sin(k * w * tie) / k);
                end

                % Ajustamos la amplitud final a lo que pidió el usuario
                % (Multiplicamos por 4/pi para ajustar la escala de Fourier o
simplemente normalizamos)
                app.y_original = app.y_original * app.amplitudField.Value;
                % Normalización simple para que el pico sea exacto a la
amplitud deseada:
                app.y_original = app.y_original / max(abs(app.y_original)) *
app.amplitudField.Value;

                 case 'Triangular'
                        app.y_original =
app.amplitudField.Value*sawtooth(w*tie,0.5);

                 case 'Sierra'
                        % Inicializamos en 0
                app.y_original = zeros(size(tie));

                % Número de armónicos a sumar.
                % 20 o 30 es suficiente para que parezca una sierra pero se
vea limpia.
                % OJO: Podrías calcular N dinámicamente para no pasar de Fs/2
                num_armonicos = 30;

                % Bucle de Síntesis Aditiva
                for k = 1:1:num_armonicos
                    % A diferencia de la cuadrada, aquí k avanza de 1 en 1
(1,2,3,4...)
                    % El signo alterna para la fase correcta, o simplemente
sumamos:
```

```
                        % La amplitud baja proporcionalmente a 1/k
                        app.y_original = app.y_original + (sin(k * w * tie) / k);
                    end

                    % La serie de Fourier de la sierra sale invertida y con
amplitud rara.
                    % La normalizamos para que tenga EXACTAMENTE la amplitud que
pide el usuario:

                    % 1. Normalizamos a 1 (dividimos por el máximo valor absoluto)
                    val_max = max(abs(app.y_original));
                    if val_max > 0
                        app.y_original = app.y_original / val_max;
                    end

                    % 2. Multiplicamos por la amplitud deseada y cambiamos signo
si quieres rampa subida/bajada
                    app.y_original = -1 * app.y_original*app.amplitudField.Value;
            end

            %Se vacía la variable de reproducción
               app.player_original = [];

            %Se grafica la señal
               app.actualizarGraficasOriginales();
        end
```

c. Grabación mediante micrófono

La función se inicializa cuando se pulsa el botón rojo (arriba se indica que se pulsa para comenzar la grabación y se vuelve a pulsar para finalizarla).

Si no se está grabando (***app.IsRecording*** es falso), se cambia el icono del botón y el valor de ***app.IsRecording***. Definimos ***Fs*** como 44100 Hz, ya que es la frecuencia estándar para la calidad de sonido de CD. Utilizamos ***app.objgr*** para almacenar la el objeto grabador con la frecuencia de muestreo ***Fs*** que había definido previamente, 16 bits (estándar) y 1 canal de audio (mono) mediante ***audiorecorder*** y empiezo a grabar con ***record***.

Si el valor de ***app.IsRecording*** es verdadero,se cambia el icono del botón y se para la grabación (***stop***). Almacenamos en ***y_original*** los datos del audio almacenado en ***objgr*** y vaciamos el objeto reproductor (***player_original***). Llamamos a la función ***actualizarGraficasOriginales()***.

Código:

```
        function GrabarButtonPushed(app, event)
        %Cuando se pulsa el botón de "Grabar", comienza la grabación

            if ~app.IsRecording
            %Si no se está grabando

                app.GrabarButton.Icon = 'icono_grabando.png';
                app.IsRecording = true;

                %La frecuencia de muestreo será 44100 Hz (estándar de calidad de
cd)
                app.Fs = 44100;

                %Características del objeto grabador
```

```
                app.objgr = audiorecorder(app.Fs, 16, 1); %16 bits (estándar) y 1
canal de audio (mono)

                %Empiezo a grabar y lo almaceno en el objeto
                record(app.objgr);

            else
            %Si se está grabando

                app.GrabarButton.Icon = 'icono_no_grabando.png';
                app.IsRecording = false;

                %Detengo la grabación
                stop(app.objgr);

                %Obtengo los datos de audio grabado (en el objeto)
                app.y_original = getaudiodata(app.objgr);

                %Vacío la variable de reproducción
                app.player_original = [];

                %Grafico la señal
                app.actualizarGraficasOriginales();
            end
        end
```

d. Reproducción de señales originales

La reproducción de señales se implementa mediante los botones de “Play” y “Pause” del extremo izquierdo de la interfaz, cada uno tiene una función que se inicializa al ser pulsados.

La función ***PlayButtonPushed*** coge el valor de ***app.y_original*** para reproducirlo cuando ***app.IsAudioReversed*** es falsa, y el vector ***app.y_original*** invertido cuando es verdadera. Guarda el correspondiente en ***audioData***.

Después, verifica que haya una señal original (puede ser cargada, generada o grabada) y una frecuencia de muestreo. Si no hay datos, se muestra una alerta que lo indica.

Si se está reproduciendo el audio procesado, paramos ese.

Comprobamos que el ***player_original*** está vacío para inicializar en él el objeto reproductor, y si lo está, mandamos a la función ***audioplayer*** los atributos ***audioData*** y ***app.Fs***. Si ya estuviera inicializado, será porque se quiere reanudar una reproducción.

Por último, comprobamos que el audio no se está reproduciendo y, o bien empezamos la reproducción, o bien la reanudamos.

Código:

```
        function PlayButtonPushed(app, event)
        %Comienza cuando pulsamos el icono "Play"

            if app.IsAudioReversed
                audioData = flipud(app.y_original); % Invierte el vector de audio
            else
                audioData = app.y_original;
            end

            %Si no hay datos de audio, se indica
```

```
            if isempty(app.y_original) || isempty(app.Fs)
                uialert(app.UIFigure,'No hay audio cargado.', 'Error');
                return;
            end

            %Si está sonando el audio procesado, lo paramos
            if ~isempty(app.player_procesada) && isplaying(app.player_procesada)
                stop(app.player_procesada);
            end

            %Si la pista de audio está vacía, la creamos (en cada función la
vacíamos para
            %actualizar el audio que queremos reproducir)
            if isempty(app.player_original)
                app.player_original = audioplayer(audioData, app.Fs);
            end

            %Si no está sonando, la reproducimos (empezamos a reproducir o
reanudamos)
            if ~isplaying(app.player_original)
                resume(app.player_original);
            end

        end
```

La función ***PauseButtonPushed*** se inicializa cuando se pulsa “Pause” en la interfaz. Se comprueba que hay un objeto reproductor inicializado en ***player_original*** y que se está reproduciendo. Si se cumplen estas dos condiciones, se pausa el audio.

Código:

```
        function PauseButtonPushed(app, event)
        %Comienza cuando pulsamos el icono de "Pause"

            %Si la pista de audio no está vacía y se está reproduciendo, se pausa
            if ~isempty(app.player_original) && isplaying(app.player_original)
                pause(app.player_original);
            end

        end
```

e. Representación de señales originales

Esta función es llamada por las funciones que se encargan de cargar, grabar o generar las señales.

Comprueba si existe una señal original y su correspondiente frecuencia de muestreo. Si no es así, lo indica.

Código:

```
        function actualizarGraficasOriginales(app)
        %Llamada desde todas las funciones que trabajan con señales originales

            if isempty(app.y_original) || isempty(app.Fs)
            %Si no hay señal a muestrear o no hay frecuencia de muestreo

                disp('no hay datos a graficar.');
                return;
            end
```

i. Tiempo

Se crea una variable ***duracion*** a partir de la longitud de la señal y su frecuencia de muestreo y se crea un vector ***t*** de tiempo.

Se grafica la señal en ese vector.

```
%TIEMPO

%Crea una variable duración (muestras/s/s)
duracion = length(app.y_original) / app.Fs;

%Crea un vector de tiempo
t = linspace(0, duracion, length(app.y_original));

%Grafico la señal con el vector t
plot(app.TiempoAxes, t, app.y_original);
title(app.TiempoAxes, 'Grafica Tiempo');
xlabel(app.TiempoAxes, 'Tiempo (s)');
ylabel(app.TiempoAxes, 'Amplitud');
grid(app.TiempoAxes, 'on');

%Ajustamos la escala del eje x a la duración
xlim(app.TiempoAxes, [0 duracion]);

%El eje y se autoajusta
ylim(app.TiempoAxes, 'auto');
```

ii. Espectrograma

Se definen los parámetros del espectrograma: ***window*** es la ventana (se hace de ***win_size*** muestras) y ***noverlap*** las muestras que se quieren solapar entre ventanas. Se define el rango para la fft y se calcula el espectrograma con la función ***spectrogram***.

Por último, se dibuja.

```
%ESPECTROGRAMA

long_senal = length(app.y_original);

%Parametros de tamaño de ventana y solapamiento
if long_senal < 1024
    win_size = long_senal;
    noverlap = long_senal/2;
else
    win_size = 1024;
    noverlap = 512;
end

%Creo la ventana de muestras
window = hamming(win_size);

%Rango de frecuencias para la resolución
nfft = 1024;

%Calculo el spectrograma (parámetros)
[~, F, T, P] = spectrogram(app.y_original, window, noverlap, nfft,
app.Fs, 'yaxis');

%Muestro el espectrograma
```

```
            imagesc(app.Espectrograma, T, F, 10*log10(P));

            axis(app.Espectrograma, 'xy');
            colormap(app.Espectrograma, 'hot');

            title(app.Espectrograma, 'Espectrograma de la Señal');
            xlabel(app.Espectrograma, 'Tiempo (s)');
            ylabel(app.Espectrograma, 'Frecuencia (Hz)');

        end
```

4. Procesado de señales

La función comienza cuando se pulsa “Procesar señal”.

Se comprueba si hay una señal original que procesar, y si no es así, se muestra una alerta. Tras esto se abre un bloque try – catch. Una estructura switch gestiona el procesado a realizar según los casos que se explican a continuación.

Código:

```
        function ProcesarsealButtonPushed(app, event)
                    %Comienza cuando se elige un tipo de procesado

            %Si no hay señal original para procesar, lo indica
            if (app.y_original == 0)
                     uialert(app.UIFigure, 'no hay señal a procesar');
                     app.opcionesdeprocesado.Value =
app.opcionesdeprocesado.Items{1};
                     return;
            end

            try
                     switch app.opcionesdeprocesado.Value
                %Para los 3 casos se calculan señales nuevas mediante el
                %resample y una nueva frecuencia de muestreo
```

Cuando finaliza el switch, se cambia el valor del toggle de elección de gráficas y se llama a la función de cambio de valor del toggle para que muestre el espectrograma de la señal original (la otra opción es la gráfica en frecuencia). Después, se llama a ***actualizarGraficasProcesadas()*** para graficar la señal procesada (generada en cada caso del switch con distintos parámetros dependiendo del caso elegido).
Si existe una señal procesada y se está reproduciendo, se detiene y se sale del if. Si existe, pero no se está reproduciendo, solo se vacía la variable donde se almacenará el objeto (***player_procesada***)

Por último, se hace el catch del try para avisar si se detectan errores.

Código:

```
                %Mandamos a graficar la señal
                app.Switch.Value = "Espectrograma";
                SwitchValueChanged(app);
                app.actualizarGraficasProcesadas();

                %Si existe un reproductor de procesado, lo paramos
                if ~isempty(app.player_procesada)
                    if isplaying(app.player_procesada)
```

```
                    stop(app.player_procesada);
                end

                %Lo vacíamos para actualizar el audio
                app.player_procesada = [];
            end

        catch ME
            uialert(app.UIFigure, ['error durante el procesado' ME.message],
'Error de Archivo');
        end
    end
```

a. Compresión

La compresión de la señal se selecciona mediante el caso “Compresion”.

El factor de compresión se obtiene haciendo ***rat(app.factordecompresion.Value)***, que racionaliza a una fracción de números enteros el valor que tendría factorCompresion.

Una vez obtenidos ***P*** y ***Q***, se comprime la señal aplicando una nueva frecuencia de muestreo (***Fs_procesada***) al mandarla tanto a las gráficas como a la reproducción.

Código:

```
            case 'Compresion'
        %Se obtienen el factor de compresión como una fración
                [P, Q] = rat(app.factordecompresion.Value);

        %Se comprime la señal en ese factor
                app.y_procesada = app.y_original;
                app.Fs_procesada = app.Fs*P/Q;
```

b. Expansión

La expansión de la señal se selecciona mediante el caso “Expansion”.

El factor de expansión se obtiene haciendo ***rat(1/app.factordeexpansion.Value)***, que racionaliza a una fracción de números enteros el valor que tendría 1/factorExpansion.

Una vez obtenidos ***P*** y ***Q***, se expande la señal aplicando una nueva frecuencia de muestreo (***Fs_procesada***) al mandarla tanto a las gráficas como a la reproducción.

Código:

```
            case 'Expansion'
        %Se obtiene el factor de expansión como una fracción
                [P, Q] = rat(1/app.factordeexpansion.Value);

        %Se expande la señal en ese factor
                app.y_procesada = app.y_original;
                app.Fs_procesada = app.Fs*P/Q;
```

c. Compresión y expansión

Se combina el método de compresión y expansión. Primero se hace un proceso exacto al de la compresión y se almacena la señal en ***y_comprimida*** y su nueva frecuencia de muestreo en ***fs_comprimida***, y después se hace un proceso de expansión a partir de estas. Finalmente, obtenemos ***y_procesada*** y ***Fs_procesada***.

Código:

```
                        case 'Compresion + Expansion'
                    %Se obtiene el factor de compresión como una fracción
                            [P_comp, Q_comp] =
rat(app.factordecompresion.Value);

                    %Se comprime la señal
                            y_comprimida = app.y_original;
                            fs_comprimida = app.Fs*P_comp/Q_comp;

                    %Se obtiene el factor de expansión como una fracción
                            [P, Q] = rat(1/app.factordeexpansion.Value);

                    %Se expande la señal comprimida
                            app.y_procesada = y_comprimida;
                            app.Fs_procesada = fs_comprimida*P/Q;
            end
```

d. Reproducción de señales procesadas

La reproducción de señales se implementa mediante los botones de “Play” y “Pause” situados encima de las gráficas de procesado de la interfaz, y, al igual que antes, cada uno tiene una función que se inicializa al ser pulsados.

La función ***PlayButtonProcesadaPushed*** coge el valor de ***app.y_procesada*** para reproducirlo cuando ***app.IsAudioReversed*** es falsa, y el vector ***app.y_procesada*** invertido cuando es verdadera. Guarda el correspondiente en ***audioData***.

Después, verifica que exista una señal procesada y una frecuencia de muestreo para esa señal. Si no hay datos, se muestra una alerta que lo indica.

Si se está reproduciendo el audio de la señal original, lo paramos.

Comprobamos que el ***player_procesada*** está vacío para inicializar en él el objeto reproductor (con la señal y su frecuencia de muestreo correspondiente). Igual que pasaba cuando reproducíamos la señal original, vemos que si ya está inicializado es porque se quiere reanudar una reproducción.

Por último, comprobamos que el audio no se está reproduciendo y, o bien empezamos la reproducción, o bien la reanudamos.

Código:

```
        function PlayButtonProcesadaPushed(app, event)
        %Comienza cuando pulsamos el icono "Play"

            if app.IsAudioReversed
                audioData = flipud(app.y_procesada); % Invierte el vector de audio
            else
                audioData = app.y_procesada;
            end

            %Si no hay datos de audio procesado, se indica
            if isempty(app.y_procesada) || isempty(app.Fs_procesada)
```

```
            uialert(app.UIFigure,'No hay señal procesada.', 'Error');
            return;
        end

        %Si está sonando el audio original, lo paramos
        if ~isempty(app.player_original) && isplaying(app.player_original)
            stop(app.player_original);
        end

        %Si la pista de audio está vacía, la creamos (en cada función la
vacíamos para
        %actualizar el audio que queremos reproducir)
        if isempty(app.player_procesada)
            app.player_procesada = audioplayer(audioData, app.Fs_procesada);
            %igual hay que borrarlo para la siguiente
        end

        %Si no está sonando, la reproducimos (empezamos a reproducir o
reanudamos)
        if ~isplaying(app.player_procesada)
            resume(app.player_procesada);
        end
    end
```

La función ***PauseButtonProcesadaPushed*** se inicializa cuando se pulsa “Pause” en la interfaz. Se comprueba que hay un objeto reproductor inicializado en ***player_procesada*** y que se está reproduciendo. Si se cumplen estas dos condiciones, se pausa el audio.

Código:

```
    function PauseButtonProcesadaPushed(app, event)
    %Comienza cuando pulsamos el icono de "Pause"

        %Si la pista de audio no está vacía y se está reproduciendo, se pausa
        if ~isempty(app.player_procesada) && isplaying(app.player_procesada)
            pause(app.player_procesada);
        end
    end
```

e. Representación de señales procesadas

Esta función es llamada por la función que se encarga de procesar las señales.

Comprueba si existe una señal procesada y su correspondiente frecuencia de muestreo. Si no es así, lo indica.

```
    function actualizarGraficasProcesadas(app)
    %Llamada desde todas la función que trabaja con señales procesadas

          if isempty(app.y_procesada) || isempty(app.Fs_procesada)
            %Si no hay señal a muestrear o no hay frecuencia de muestreo

                disp('no hay datos a graficar.');
                return;
          end
```

i. Tiempo

Se crea una variable ***duracion*** a partir de la longitud de la señal y su frecuencia de muestreo y se crea un vector ***t*** de tiempo.

Se grafica la señal en ese vector.

```
%TIEMPO

            %Crea una variable duración (muestras/s/s)
               duracion = length(app.y_procesada) / app.Fs_procesada;

            %Crea un vector de tiempo
               t = linspace(0, duracion, length(app.y_procesada));

               %Grafico la señal con el vector t
               plot(app.GraficaTiempoProcesada, t, app.y_procesada);
               title(app.GraficaTiempoProcesada, 'Grafica Procesada Tiempo');
               xlabel(app.GraficaTiempoProcesada, 'Tiempo (s)');
               ylabel(app.GraficaTiempoProcesada, 'Amplitud');
               grid(app.GraficaTiempoProcesada, 'on');

            %Ajustamos la escala del eje x a la duración
               xlim(app.GraficaTiempoProcesada, [0 duracion]);

            %El eje y se autoajusta
               ylim(app.GraficaTiempoProcesada, 'auto');
```

ii. Espectrograma

Se definen los parámetros del espectrograma. ***window*** es la ventana y ***noverlap*** las muestras que se quieren solapar entre ventanas. Se define el rango para la fft y se calcula el espectrograma con la función ***spectrogram***.

Por último, se dibuja.

```
            %ESPECTROGRAMA

            %Parametros de tamaño de ventana y solapamiento
            window = hamming(1024);
            noverlap = 512;

            %Rango de frecuencias para la resolución
            nfft = 1024;

            %Calculo el spectrograma (parámetros)
            [~, F, T, P] = spectrogram(app.y_procesada, window, noverlap, nfft,
app.Fs_procesada, 'yaxis');

            %Muestro el espectrograma
            imagesc(app.EspectrogramaProcesado, T, F, 10*log10(P));

            axis(app.EspectrogramaProcesado, 'xy');
            colormap(app.EspectrogramaProcesado, 'hot');

            title(app.EspectrogramaProcesado, 'Espectrograma de la Señal
Procesada');
            xlabel(app.EspectrogramaProcesado, 'Tiempo (s)');
            ylabel(app.EspectrogramaProcesado, 'Frecuencia(Hz)');
        end
```

5. Exportación de señales

Podemos exportar las señales (los audios) para guardarlos en el sistema pulsando los botones “Guardar” o “Guardar procesada”.

a. Exportación de señales originales

Esta función se inicializa cuando pulsamos el botón “Guardar” de la interfaz.

Si no hay ninguna señal original (***y_original***), salta una ventana en la interfaz que indica que no hay ningún dato para guardar.

Si la señal original sí que existe, se crea una variable para el nombre por defecto (***nombre_defecto***) y el formato (***filter***) que tendrá el archivo cuando salte la ventana de diálogo para almacenar el archivo con ***uiputfile***. Se comprueba si se pulsa “Cancelar”, y si es así, la función finaliza.

Si el usuario selecciona “Guardar Audio”, se combina la ruta con ***fullfile*** pasandole el nombre y la ruta como atributos.

Finalmente, se hace un bloque try – catch para crear el archivo. Si se logra, se indica en una ventana, y si no, se muestra una alerta con el error.

```
        function GuardarButtonPushed(app, event)
        %Comienza cuando pulsamos "Guardar procesada"

            %Si no hay señal original, salta un aviso
            if isempty(app.y_original) %si no hay señal a guardar saco un aviso
                uialert(app.UIFigure, 'No hay señal original para guardar.',
'Aviso');
                return;
            end

            %Se indica un nombre por defecto para el audio y un formato de audio
            nombre_defecto = 'audio_original.wav';
            filter = {'*.wav', 'Archivo WAV (*.wav)'};

            %Se crea una ventana para guardar el audio (con nombre y formato)
            [file, path] = uiputfile(filter, 'Guardar Audio', nombre_defecto);

            %Si se pulsa "Cancelar", se sale
            if isequal(file, 0)
                return;
            end

            %Combina la ruta y el archivo
            fullFileName = fullfile(path, file);

            try
                %Crea el archivo donde se haya elegido y lo indica
                audiowrite(fullFileName, app.y_original, app.Fs);
                uialert(app.UIFigure, 'Archivo guardado correctamente.', 'Éxito');
            catch ME
                uialert(app.UIFigure, ['Error al guardar: ' ME.message], 'Error');
            end
        end
```

b. Exportación de señales procesadas

Si no hay ninguna señal procesada (***y_procesada***), salta una ventana en la interfaz que indica que no hay ningún dato para guardar.

Si la señal procesada existe, al igual que antes, se crea una variable para el nombre por defecto (***nombre_defecto***) y el formato (***filter***) que tendrá el archivo cuando salte la ventana de diálogo para almacenar el archivo con ***uiputfile***. Se comprueba si se pulsa "Cancelar", y si es así, la función finaliza.

Si el usuario selecciona "Guardar Audio", se combina la ruta con ***fullfile*** pasandole el nombre y la ruta como atributos.

Finalmente, se hace un bloque try – catch para crear el archivo. Si se logra, se indica en una ventana, y si no, se muestra una alerta con el error.

```
        function GuardarProcesadaButtonPushed(app, event)
        %Comienza cuando pulsamos "Guardar procesada"

            %Si no hay señal procesada, salta un aviso
            if isempty(app.y_procesada)
                uialert(app.UIFigure, 'No hay señal procesada para guardar.',
'Aviso');
                return;
            end

            %Se indica un nombre por defecto para el audio y un formato de audio
            nombre_defecto = 'audio_procesado.wav';
            filter = {'*.wav', 'Archivo WAV (*.wav)'};

            %Se crea una ventana para guardar el audio (con nombre y formato)
            [file, path] = uiputfile(filter, 'Guardar Audio', nombre_defecto);

            %Si se pulsa "Cancelar", se sale
            if isequal(file, 0)
                return;
            end

            %Combina la ruta y el archivo
            fullFileName = fullfile(path, file);

            try
                %Crea el archivo donde se haya elegido y lo indica
                audiowrite(fullFileName, app.y_procesada, app.Fs_procesada);
                uialert(app.UIFigure, 'Archivo guardado correctamente.', 'Éxito');
            catch ME
                uialert(app.UIFigure, ['Error al guardar: ' ME.message], 'Error');
            end
        end
```

6. Inversión de ejes

Cuando se pulsa el botón "Invertir Ejes", comienza la función.

Cambia el valor de ***app.IsAudioReversed*** a su contrario, y crea una variable que almacena la dirección del eje x de la gráfica de tiempo de la señal original. Si esa variable es igual a la cadena "normal", revierte los ejes de la gráfica de tiempo y el espectrograma de la señal

origina. Comprueba si existe una señal procesada, y si es así, también invierte los ejes de sus gráficas.

Hace lo contrario si la cadena es “reverse”, es decir, hace que la dirección de los ejes sea “normal”.

Por último, comprueba si las señales se están reproduciendo. Si es asi, las para y vacía la variable del objeto reproductor. De esta forma, si se vuelve a pulsar el botón de “Play”, se reproducirá la señal invertida.

```
function InvertirejesButtonPushed(app, event)
    % Button pushed function: InvertirEjesButton
    % --- INVERSIÓN DEL EJE X (Tiempo) ---
    app.IsAudioReversed = ~app.IsAudioReversed;

    % 1. Verificar el estado actual del eje X (XDir)
    current_xdir = app.TiempoAxes.XDir;

    % 2. Invertir el eje X
    if strcmp(current_xdir, 'normal')
        % Si está normal, lo invertimos
        app.TiempoAxes.XDir = 'reverse';
        app.Espectrograma.XDir = 'reverse';
        if ~isempty(app.y_procesada)
            app.GraficaTiempoProcesada.XDir = 'reverse';
            app.EspectrogramaProcesado.XDir = 'reverse';
        end
    else
        % Si está invertido, lo volvemos a normal
        app.TiempoAxes.XDir = 'normal';
        app.Espectrograma.XDir = 'normal';
        if ~isempty(app.y_procesada)
            app.GraficaTiempoProcesada.XDir = 'normal';
            app.EspectrogramaProcesado.XDir = 'normal';
        end
    end

    if ~isempty(app.player_original) && isplaying(app.player_original)
        stop(app.player_original);
    end

    app.player_original = [];

    if ~isempty(app.player_procesada) && isplaying(app.player_procesada)
        stop(app.player_procesada);
    end

    app.player_procesada = [];

end
```

7. Cambio de gráfica (Espectrograma – Frecuencia)

Esta función comienza cuando cambia el valor del toggle para elegir si queremos ver la gráfica en frecuencia o el espectrograma. Por defecto se ve el espectrograma (se ve en las funciones de actualizar gráficas).

Si no hay señal original, se indica.

Después, se coge el valor del toggle y se guarda en ***value***. Se limpian las gráficas.

Si el valor del toggle es igual a “Frecuencia”, se calcula la transformada de Fourier de la señal original y se representa. Se comprueba si existe una señal procesada, y si se da el caso, se hace también su transformada y se representa.

Si el valor del toggle es “Espectrograma”, se hace lo mismo que cuando graficamos las señales inicialmente.

```
        function SwitchValueChanged(app, event)

            if isempty(app.y_original)
                uialert(app.UIFigure, 'no hay señal que mostrar');
            end

            value = app.Switch.Value;
            cla(app.Espectrograma, 'reset');
            cla(app.EspectrogramaProcesado, 'reset');

            if (value == "Frecuencia")

                %FRECUENCIA

                %Calculo la longitud de la señal (número de muestras)
                L = length(app.y_original);

                %Calculo la transformada de Fourier
                Y_fft = fft(app.y_original);

                %Ordena los datos
                Y_shifted = fftshift(Y_fft);

                %Obtiene la señal normalizada
                P2 = abs(Y_shifted/L);

                %Crea un vector de un ancho de banda de la frecuencia de muestreo
(-Fs/2, Fs/2)
                f_vec = (-L/2 : L/2 - 1) * (app.Fs/L);

                %Grafico la señal normalizada con f_vec
                plot(app.Espectrograma, f_vec, P2);
                title(app.Espectrograma, 'Grafica Frecuencia (Espectro)');
                xlabel(app.Espectrograma, 'Frecuencia (Hz)');
                ylabel(app.Espectrograma, 'Amplitud');
                grid(app.Espectrograma, 'on');

                %Ajustamos el eje x para que vaya de -Fs/2 a Fs/2
                xlim(app.Espectrograma, [-app.Fs/2 app.Fs/2]);

                %El eje y se autoajusta
                ylim(app.Espectrograma, 'auto');

                if ~isempty(app.y_procesada)
                    %FRECUENCIA
                       L = length(app.y_procesada);
                       Y_fft = fft(app.y_procesada);
                    Y_shifted = fftshift(Y_fft);
```

```
                    P_bilateral = abs(Y_shifted/L);
                    f_vec = (-L/2 : L/2 - 1) * (app.Fs_procesada / L);


                    plot(app.EspectrogramaProcesado, f_vec, P_bilateral);
                    title(app.EspectrogramaProcesado, 'Grafica Frecuencia
Procesada (Espectro)');
                    xlabel(app.EspectrogramaProcesado, 'Frecuencia (Hz)');
                ylabel(app.EspectrogramaProcesado, 'Amplitud');
                    grid(app.EspectrogramaProcesado, 'on');
                    xlim(app.EspectrogramaProcesado, [-app.Fs_procesada/2,
app.Fs_procesada/2]);
                    ylim(app.EspectrogramaProcesado, 'auto');
                end
            end

            if (value == "Espectrograma")

                %ESPECTROGRAMA

                long_senal = length(app.y_original);

                %Parametros de tamaño de ventana y solapamiento
                if long_senal < 1024
                    win_size = long_senal;
                    noverlap = long_senal/2;
                else
                    win_size = 1024;
                    noverlap = 512;
                end

                %Creo la ventana de muestras
                window = hamming(win_size);

                %Rango de frecuencias para la resolución
                nfft = 1024;

                %Calculo el spectrograma (parámetros)
                [~, F, T, P] = spectrogram(app.y_original, window, noverlap, nfft,
app.Fs, 'yaxis');

                %Muestro el espectrograma
                imagesc(app.Espectrograma, T, F, 10*log10(P));

                axis(app.Espectrograma, 'xy');
                colormap(app.Espectrograma, 'hot');

                title(app.Espectrograma, 'Espectrograma de la Señal');
                xlabel(app.Espectrograma, 'Tiempo (s)');
                ylabel(app.Espectrograma, 'Frecuencia (Hz)');

                if ~isempty(app.y_procesada)
                    %ESPECTROGRAMA

                    %Parametros de tamaño de ventana y solapamiento
                    window = hamming(1024);
                    noverlap = 512;

                    %Rango de frecuencias para la resolución
```

```
                    nfft = 1024;

                    %Calculo el spectrograma (parámetros)
                    [~, F, T, P] = spectrogram(app.y_procesada, window, noverlap,
nfft, app.Fs_procesada, 'yaxis');

                    %Muestro el espectrograma
                    imagesc(app.EspectrogramaProcesado, T, F, 10*log10(P));

                    axis(app.EspectrogramaProcesado, 'xy');
                    colormap(app.EspectrogramaProcesado, 'hot');

                    title(app.EspectrogramaProcesado, 'Espectrograma de la Señal
Procesada');
                    xlabel(app.EspectrogramaProcesado, 'Tiempo (s)');
                    ylabel(app.EspectrogramaProcesado, 'Frecuencia (Hz)');
                end
            end
        end
```